

JAVASCRIPT

CARACTERÍSTICAS

- Es Interpretado y compilado Just - In - Time
- Es multiparadigma
- Tipado débil y dinámico
- Asíncrono y dirigido a eventos
- Multiplataforma
- Orientado a objetos (basado en prototipos)

ESTRUCTURAS DE CONTROL

CONDICIONALES

if...else if...else: Evalúa una condición inicial, depende de la condición se ejecuta el código después de if, después de else if o después de else: ejemplo: `if (x = 5){console.log("es 5");}else{"diferente de 5"}`

switch: Evalúa una expresión y compara su valor para múltiples casos. Sintaxis: `let x = "cadena"; switch (x) { case "cad": "código a ejecutar"; break; case "cade": "ejecuta esto" break; default: "caso default"; }`

Operador ternario: Un atajo excelente para condiciones simples de tipo if-else. Su sintaxis es: `condición ? verdadero : falso.`

BUCLES

for: Se usa cuando conoces de antemano el número de repeticiones.. Sintaxis: `for (inicialización; condición; actualización) { // Código a ejecutar }`

while: Se usa cuando el número de repeticiones depende de una condición dinámica. `while (condición) { // Código a ejecutar }`

do...while: Igual que el while, pero garantiza que el código se ejecute al menos una vez. `do { // Código a ejecutar } while (condición);`

TIPOS DE DATOS

PRIMITIVOS

Son valores inmutables y se comparan directamente por su valor. Algunos ejemplos: `let nombre = "Antony"; let edad = 22; let activo = true;`

REFERENCIAS

Son estructuras complejas y mutables. Al copiarlos, compartes el acceso al mismo espacio de memoria. Ejemplo: `Object: Colección de propiedades con clave y valor. let usuario = { nombre: "Ana", edad: 28 };`

FUNCIONES

DECLARADAS

Funciones Declaradas: Se definen con la palabra clave `function` y un nombre obligatorio.

EXPRESADAS

Funciones Expresadas: Se definen asignando una función (normalmente anónima) a una variable.

DE FLECHA

Funciones de Flecha (Arrow Functions): Son una alternativa compacta a las funciones expresadas. Usan la sintaxis `=>`, no tienen su propio contexto para la palabra clave `this` y permiten retornos implícitos de una sola línea.

JAVASCRIPT

OBJETOS

Objetos: Estructuras que representan elementos del mundo real mediante colecciones de propiedades (clave-valor).

```
const coche = { marca: "Toyota", modelo: "Corolla", año: 2024 };
```

ARRAYS

Arrays: Listas ordenadas de elementos indexados numéricamente que comienzan en la posición 0.

```
const frutas = ["manzana", "plátano", "naranja"];
```

MANIPULACIÓN DEL DOM

El DOM (Document Object Model) es una interfaz de programación que representa el documento HTML como un árbol de nodos, permitiendo a JavaScript modificar la estructura, el estilo y el contenido de una página web.

```
// Ejemplo: Seleccionar un elemento de la página y cambiar su texto
const titulo = document.querySelector("#mi-titulo");
titulo.textContent = "¡Hola Mundo desde el DOM!";
```

ASINCRONÍA

La asincronía en JavaScript permite ejecutar tareas de larga duración (como pedir datos a un servidor) en segundo plano, evitando que la página web se congele mientras espera el resultado.

Callbacks: Funciones que se pasan como argumentos a otra función para ejecutarse una vez que finaliza la tarea asíncrona.

Promesas: Objetos que representan el éxito o el fracaso futuro de una operación asíncrona mediante los métodos `.then()` y `.catch()`.

Async/Await: Sintaxis moderna que trabaja sobre las promesas, permitiendo escribir código asíncrono con una estructura que se lee como si fuera síncrono. [1]